Freight Rate Prediction

Machine Learning Engineer Assessment - report

Candidate: Rahul | Submitted to: Spotter | Holdout (train Jan-Aug to test Sep-Oct): RMSE $52.56, MAPE 1.57% (clean loads)

Executive summary

I built a LightGBM model that predicts rate per mile (posted_rate / distance) and multiplies by distance, trained only on inputs that exist for every scored load: pickup/delivery city and coordinates, distance, equipment, weight and date. On an honest forward holdout (train January-August, test September-October 2025) it reaches RMSE $52.56, MAE $36.23 and MAPE 1.57% on clean test loads, against $295.92 / 9.27% for a median-rate baseline. The final model is refit on all clean January-October data (3 seeds averaged) and passes the provided scorer for all 12,000 validation loads and the 31 December chart rows.

Three findings drive this result, and each is a reason a naive pipeline fails here:

- **quote_signal is an unstable leak and is excluded.** In some training months it is an almost exact copy of the target rate-per-mile (correlation +0.996), in others its mirror image 4.15 - rpm (-0.994), and in August pure noise. A label-free fingerprint shows November and December - the scored period - are in the noise regime, so any model trusting the quote is harmed exactly where it is scored.

- **market_index is excluded.** It is 97% explained by the calendar (annual Fourier terms + day-of-week). Used raw it degrades the forward holdout badly (RMSE $63.91 to $91.83) because its autumn levels collide with winter levels. Calendar features carry the same signal and extrapolate. Neither field exists in the December chart inputs, which confirms they cannot be production features.

- **About 1.4% of training labels are corrupted and are dropped.** The rate-per-mile histogram has empty gaps; outside them, rates appear multiplied or divided by roughly 2-6.5x. Training keeps only loads with rate-per-mile in [$1.50, $3.70].

1 Data overview

File	Rows	Dates	Role
train_test.csv	48,000	2025-01-01 to 2025-10-31	Labelled development data (posted_rate)
validation.csv	12,000	2025-11-01 to 2025-12-31	Loads to predict (load_id TE-000001...TE-012000)
validation_predictions_template.csv	12,000	-	Template filled to validation_predictions.csv
december_chart_inputs.csv	31	2025-12-01 to 2025-12-31	Lexington to Fort Wayne, 360 mi, Dry Van, 32,000 lb; only date varies

Each load has pickup/delivery city with coordinates, distance, equipment (Dry Van 57%, Reefer 25%, Flatbed 18%), weight, date, market_index and quote_signal; the target is posted_rate (mean $2,374, median $2,031). The task is explicitly a future-prediction task: every scored date is after every labelled date. Coordinates are constant per city and identical in the pickup and delivery roles, and reported distance is consistent with the given coordinates (haversine correlation 0.9995, median ratio 1.18). The coordinates do not match real-world geography for these city names, so I treat them as the given abstract locations and never geocode externally - external coordinates would break consistency with the supplied distances.

I model rate per mile (rpm = posted_rate / distance, clean median about $2.15/mi) rather than the raw rate: distance alone correlates 0.91 with the rate, and rpm has interpretable structure in distance, equipment,

weight, geography and calendar effects that a model can learn without also having to rediscover proportionality to miles.

2 Exploratory findings and data quality

2.1 Target structure and corrupted labels

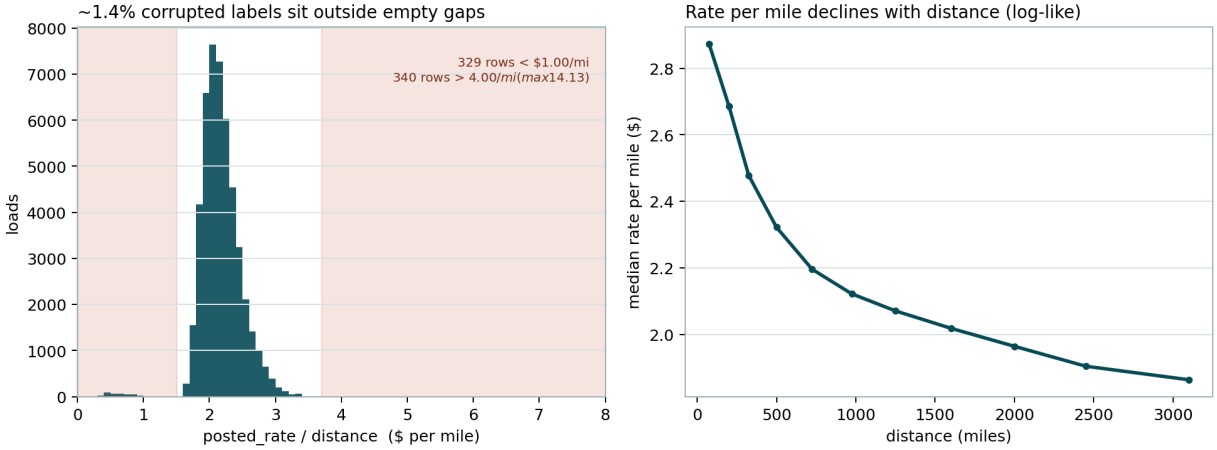


Figure 1 - Left: rate-per-mile histogram. The legitimate mass sits between $1.60 and $3.60/mi; 329 loads fall below $1.00 and 340 above $4.00 (maximum $14.13/mi), separated by near-empty gaps - these are corrupted labels, not a heavy tail. Right: rate per mile declines smoothly with distance (median $2.87/mi under 150 miles to $1.86/mi beyond 2,700 miles); a log-distance fit already explains 63% of clean rpm variance.

The corrupted rows are spread evenly across months, lanes and equipment, and their quote_signal values are normal; the inflation/deflation factors (roughly 2-6.5x each way) look applied to the rate after generation. Dropping rpm outside [$1.50, $3.70] removes 677 rows (1.41%) and leaves 47,323 clean training loads. A Huber-loss model trained on unfiltered data did not beat the clean-filter model on the holdout, so the explicit filter is kept. Holdout metrics are reported on clean test rows (model quality) and on all rows (what an operator would see, dominated by the same corruption in test labels).

2.2 quote_signal: a leaking feature with switching regimes

quote_signal is centred near $2/mi, the scale of the target rate-per-mile itself. On clean rows, month by month, it is one of three things - switches occur exactly at month boundaries:

Training month	corr(quote, rpm)	Diagnostic	Regime
Jan, Feb, Mar, Jun, Sep	+0.996 to +0.997	std(rpm - quote) = $0.022/mi	Copy: quote = rpm + noise
Apr, May, Jul, Oct	-0.993 to -0.994	std(rpm + quote) = $0.030/mi; mean sum = 4.150	Mirror: quote = 4.15 - rpm + noise
Aug	-0.020	quote mean 2.053, std 0.223, independent of everything	Noise

In copy months the quote essentially hands over the answer; in mirror months the answer is recovered only after the transform 4.15 - quote. The scored months have no labels, but the regime can still be read off without them: true rate-per-mile falls with distance, so in informative months corr(quote, log distance) is about -0.80 (copy) or +0.81 (mirror), while a noise quote gives about 0 (August: +0.018). For the validation loads this fingerprint is +0.009 in November and +0.013 in December, and the quote distribution there (mean 2.051, std 0.220, flat across distance bands and weeks) matches the August noise generator almost exactly. Conclusion: in the scored period quote_signal carries no information.

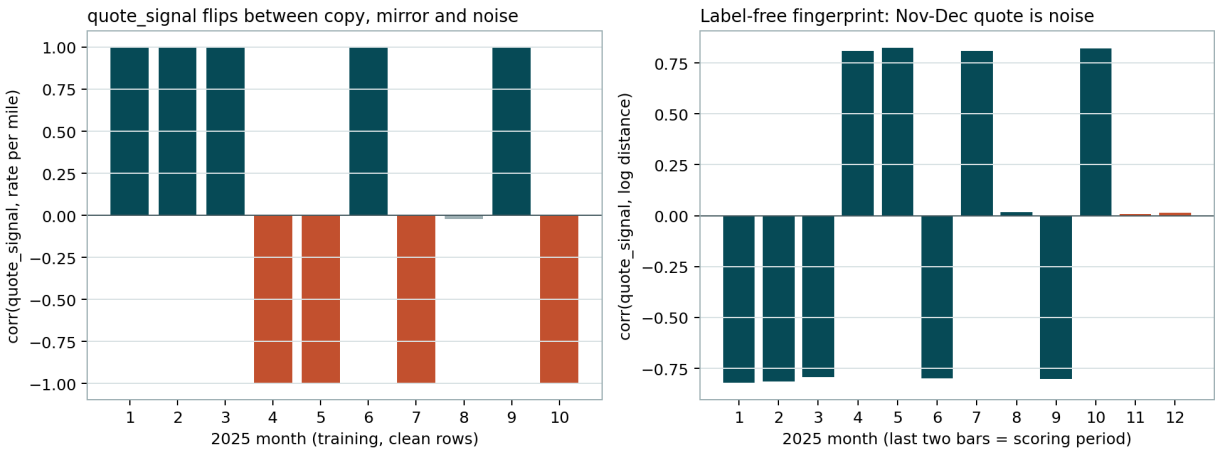


Figure 2 - Left: monthly correlation between quote_signal and true rate-per-mile on clean training rows (copy / mirror / noise regimes). Right: the label-free fingerprint corr(quote, log distance); the last two bars are the scored months and sit at zero - the quote is noise there.

The cost of getting this wrong is large. Even the naive predictors illustrate it on clean holdout rows:

Naive predictor	September RMSE (copy month)	October RMSE (mirror month)
quote x distance	$29.91	$481.75
(4.15 - quote) x distance	$530.62	$40.44

A gradient-boosted model given the raw quote does better than the naive rules but still degrades the future holdout overall ($63.91 to $77.51 RMSE). Since the scored regime is noise, the only robust choices are to detect-and-transform (fragile, and impossible for the December file which has no quote column at all) or to exclude the feature. I exclude it.

> This is also why random cross-validation is dangerous here: on a random 80/20 split, adding the quote (with month) appears to improve RMSE from $60.69 to $40.31, because train and test share the same monthly regimes. That improvement is leakage about the past, not skill about the future.

2.3 market_index duplicates the calendar

market_index is a daily-level series (within-day std 0.025) with a smooth annual hump (monthly means 0.89-1.30) and a weekly sawtooth (Thursday peak). A calendar regression (annual Fourier harmonics K=3 + day-of-week) explains 97.1% of its daily variance, and daily distance-adjusted rates follow the same calendar shape (daily correlation 0.68). At row level it can look predictive, but on the future holdout adding it raw hurt badly ($63.91 to $91.83 RMSE; market-instead-of-calendar: $147.81): its September-October levels (about 0.9) resemble January levels, yet rates at those dates differ - a tree keys on the level and extrapolates to the wrong season. A detrended version (deviation from its own calendar fit) also added nothing ($53.00 to $55.32). I therefore model seasonality directly with Fourier terms and day-of-week, which are defined for future dates and present in every input file.

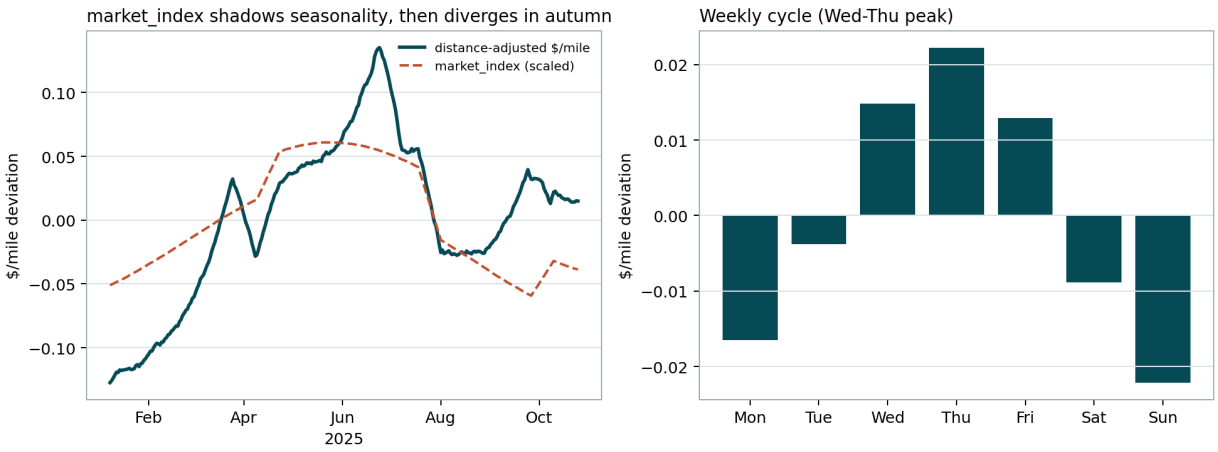


Figure 3 - Left: 14-day smoothed distance-adjusted rate per mile (solid) and scaled market_index (dashed): the same annual shape until autumn, where the index falls while rates recover - one more reason not to extrapolate with it. Right: weekly cycle, Wednesday-Thursday peak, weekend trough (about +/- $0.02/mi, i.e. roughly 2%).

2.4 Other data-quality issues

Issue	Evidence	Treatment
Negative weight	292 train rows (0.61%), 145 validation (1.21%); |weight| spans exactly 5,000-47,500 lb, same distribution as positives; rpm rises with |weight| on those rows just as elsewhere	Sign error: use abs(weight)
Missing weight	0.62% train, 1.38% validation; roughly uniform over time; label distribution similar when missing	Missingness indicator + LightGBM native NaN handling
Missing market_index	0.78% train, 2.08% validation	Irrelevant after exclusion (Section 2.3)
December file lacks coordinates, market_index, quote_signal	Only 7 columns supplied; Lexington and Fort Wayne appear throughout training with fixed coordinates	Look up the two cities' fixed coordinates; model uses no market/quote features by design
Unseen cities in scoring set	8 cities (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego) on 1,447 loads (12.1%) never appear in training	Price from coordinates (city effects are ~75% explained by latitude); target encodings fall back to the global mean; validated in Section 3
Distance jitter within a lane	Same lane distances vary with CV about 1.8%; rate responds to the reported distance; haversine from coordinates is jitter-free	Provide both reported distance (and log) and haversine/ratio features

2.5 What actually drives the rate (clean rows)

- Distance: median rpm falls from $2.87/mi (under 150 mi) to $1.86/mi (2,700-3,500 mi), close to logarithmic.

- Equipment: mean rpm Dry Van $2.09, Flatbed $2.26, Reefer $2.36.

- Weight: distance-adjusted rpm rises monotonically from -$0.099/mi (<=15k lb) to +$0.069/mi (>39k lb).

- Geography: city effects span about +/- $0.08/mi; pickup and delivery effects for the same city correlate 0.975; a fit on latitude alone explains 74-78% of city-effect variance (southern cities pricier, about -$0.007/mi per degree of latitude) - the key to pricing unseen cities.

- Calendar: distance-adjusted monthly rpm moves from -$0.117/mi in January to +$0.105/mi in June, with a weekly Wednesday-Thursday peak. Extra annual harmonics matter: moving from Fourier K=3 to K=4-6 cut holdout RMSE from about $64 to about $54, i.e. the seasonal curve is not a single sine wave.

3 Validation design (train/test split approach)

The labelled window (Jan-Oct) and the scored window (Nov-Dec) are disjoint in time, so I validate the way the model will be used: **train on 2025-01-01 to 2025-08-31 (37,944 clean loads), test on 2025-09-01 to 2025-10-31 (9,523 loads)**. This single forward holdout drove every modelling decision (feature inclusion, Fourier order, target encoding, hyperparameters). It deliberately contains one copy-regime month (September) and one mirror-regime month (October) for quote_signal, so leaky features are exposed rather than rewarded. Random splits were used only to demonstrate the leakage illusion (Section 2.2) - never for decisions - because they share months, lanes and regimes between train and test.

Two secondary checks support the same conclusions. First, a **city holdout stress test**: retraining with 8 random cities completely removed, September-October loads touching those unseen cities score MAPE 1.82% versus 1.62% on seen-city loads - a small, acceptable penalty that confirms coordinates generalise to the 8 genuinely new scoring cities. Second, **month-level reporting** of the main holdout (September MAPE 1.49%, October 1.65%) shows no single-month failure hiding inside the average. Target encodings inside the holdout are computed from the training window only (out-of-fold over 5 chronological folds for training rows), so no encoding sees a test label - or its own row's label.

Metrics are computed on posted_rate in dollars. "Clean" metrics exclude test rows with corrupted labels (Section 2.1) and measure model quality; "all rows" metrics include them and are dominated by a handful of 3-6x label errors (RMSE $630 for every model tried, including the leakage-free ones), so model selection used clean RMSE/MAE/MAPE with all-rows MAE as a sanity check.

4 Model

Final model: LightGBM regression on clean rate-per-mile, predictions multiplied by distance and rounded to cents. Three seeds (1, 7, 42) are averaged. Parameters: learning rate 0.03, 63 leaves, min 40 rows per leaf, feature and bagging fractions 0.9, 1,500 rounds - chosen on the forward holdout and then kept fixed; larger trees and per-equipment models were worse, Huber loss and log-target made no difference once corrupted labels are removed, and a structured Ridge model ($88.16 RMSE) confirmed the response surface is usefully nonlinear without being exotic.

Features: distance, log distance, haversine miles, distance/haversine ratio; equipment; weight and its missing indicator; pickup/delivery latitude/longitude, latitude/longitude sums and differences; day-of-week (raw, cyclic, weekend flag); annual Fourier harmonics K=6 (swept 3-8: K=4-8 all similar, K=3 clearly worse); and smoothed out-of-fold target encodings of pickup and delivery city (50-count shrinkage to the global mean; unseen cities take the global mean and lean on coordinates). Lane-level target encodings were tried and dropped (no gain beyond city effects, and >4,000 sparse lanes). quote_signal and market_index are absent by design (Section 2). Largest gain shares on the holdout fit: haversine 39%, distance 23%, equipment 19%, first annual harmonic 5%, weight 4% - consistent with the exploratory structure.

4.1 Development experiments (same forward holdout, clean test rows)

Configuration	RMSE ($)	MAE ($)	MAPE (%)
Naive: median rpm x distance	295.92	203.69	9.27
LightGBM base, Fourier K=3	63.91	42.46	1.78
+ raw market_index	91.83	63.48	2.64
market_index instead of Fourier terms	147.81	114.92	4.81
+ quote_signal	77.51	52.79	2.28
Ridge on structured basis (sanity check)	88.16	60.55	2.60
Fourier K=4-6	about 53-54	about 37	about 1.58
Final: K=6 + out-of-fold city encodings	52.56	36.23	1.57

4.2 Final holdout results

Holdout slice	Rows	RMSE ($)	MAE ($)	MAPE (%)
Sep-Oct, clean	9,379	52.56	36.23	1.57
September only	-	50.63	34.60	1.49
October only	-	54.35	37.80	1.65
Sep-Oct, all rows (incl. corrupted labels)	9,523	629.90	92.36	4.10

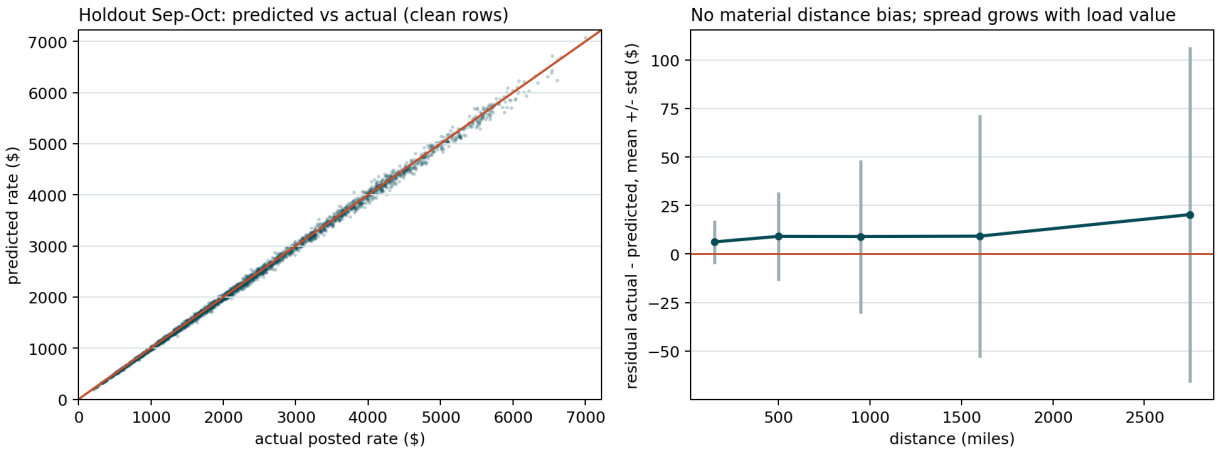


Figure 4 - Holdout diagnostics. Left: predicted vs actual on clean September-October loads. Right: residuals by distance band - means stay near zero; spread grows with load value, as expected when errors are roughly proportional. Remaining clean error (about $0.045/mi residual std) includes genuine day-level shocks: daily mean residuals vary by about $0.025/mi beyond any calendar pattern, which no trustworthy same-day feature in this data can predict (the only candidates were the two excluded fields).

5 Submission files

validation_predictions.csv contains exactly the template's 12,000 load_ids in order with positive predicted_rate values (range $185-$6,719, median $1,982). outputs/december_predictions.csv is the December input file with predicted_rate filled and its seven original columns unchanged. Both pass the provided scorer:

```
Validated 12,000 final predictions. / Validated 31 fixed December predictions. / Created chart:
outputs/candidate_december.png
```

6 December 2025 fixed-lane chart

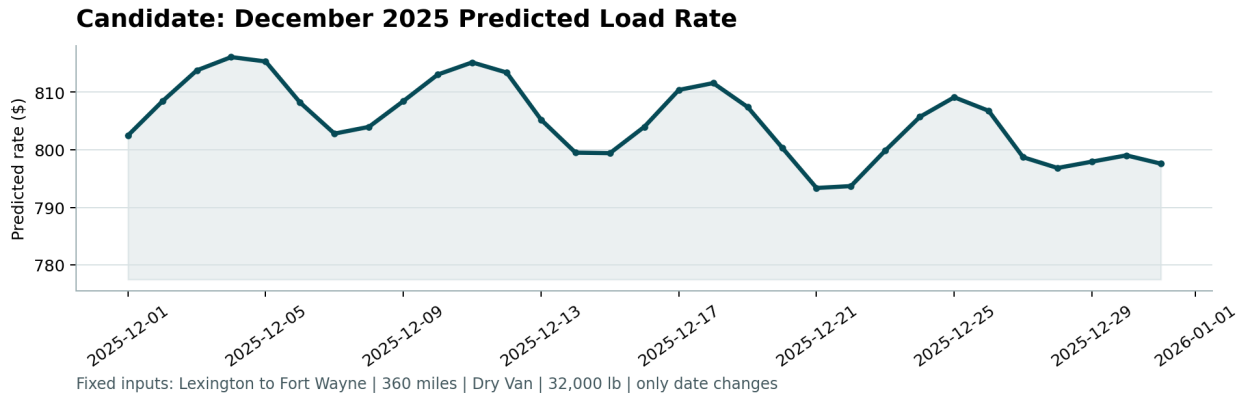


Figure 5 - candidate_december.png produced by the provided score.py: predicted rate for the fixed Lexington to Fort Wayne load (360 miles, Dry Van, 32,000 lb) for each day of December 2025.

How it was produced: the same final model used for the 12,000 loads. The December file supplies only city names, distance, equipment, weight and date; coordinates for Lexington (36.99, -85.00) and Fort Wayne (41.32, -85.36) come from their fixed training values, and no market_index or quote_signal value is needed or invented. Every modelling choice in Sections 2-4 was partly made so that this file can be priced by exactly the same model as the main task, with no imputation story.

Reading the chart: predicted rates run $793-$816, i.e. $2.20-$2.27/mi. That brackets this lane's own training history (32 Lexington to Fort Wayne loads in Jan-Oct, 21 of them Dry Van with median $2.23/mi), sitting slightly below the all-year centre because the learned annual cycle eases into winter. Within the month the weekly cycle dominates: peaks land on Thursdays (Dec 4: $816, Dec 11: $815, Dec 18: $812, Dec 25: $809) and troughs on Sundays/Mondays (Dec 21-22: $793), a swing of about $13-23 (2-3%) that matches the Wednesday-Thursday premium measured across all of 2025. There are no spikes or discontinuities: the only varying inputs are the annual Fourier terms (smooth by construction and periodic, so late December connects sensibly to the January part of the curve) and day-of-week. Two honest caveats: the training window contains no Thanksgiving/Christmas/New Year periods, so any holiday-specific capacity effects in the true process are not represented - the December path is a smooth seasonal extrapolation, not a holiday model; and day-level market shocks (about +/- $9 on this lane at one std, from holdout daily residuals) would move real quotes around this path.

7 Limitations and next steps

- Unpredictable daily shocks (about $0.025/mi std) set a floor on achievable error with these features; a production system would watch for a trustworthy real-time market signal, with regime monitoring, rather than trusting a field like quote_signal blindly.

- Holiday effects in late November/December are unlearnable from a Jan-Oct window; if Spotter has prior-year data, a holiday calendar feature is the first thing I would add and re-validate.

- The scored labels presumably contain the same ~1.4% corruption; predictions target the clean conditional centre, which is the right target for MAE/MAPE and near-optimal for RMSE.

- Unseen-city loads carry a small measured penalty (MAPE 1.82% vs 1.62% in stress test); lane-specific history would close it as those cities accumulate loads.

- Validation rests on one forward window plus the city stress test; with more time I would add rolling-origin folds (e.g. test Jul-Aug, Aug-Sep) to confirm stability of the same choices, and a prediction interval model (quantile LightGBM) for operational use.

8 Reproducibility

Code (GitHub repository root): src/features.py (cleaning, features, encodings, December frame), src/analyze.py (regenerates every EDA number and Figures 1-3 into outputs/), src/train.py (forward-holdout evaluation incl. Figure 4, then the final 3-seed fit), src/predict.py (submission files), and the provided

score.py (validation + Figure 5). Run: pip install -r requirements.txt, then python src/analyze.py, python src/train.py (about 2 minutes), python src/predict.py, and the score.py command shown in the README. Seeds are fixed; LightGBM threading can move the third decimal.

Time spent: within the requested 8 working hours, with most of it on the regime/leakage analysis - the modelling itself is deliberately conventional once the data-generating traps are understood.